

Source Code

The implementation is a single Python package, `src/`, organized along the three architectural components. Each teammate owns one branch; Person C owns the shared scaffolding (data pipeline, fusion, training/eval infra).

Annotated structure

```
src/
├─ config.py          # Single source of truth: paths, hyperparameters,
│                     #   class count, seed. Imported by every other module.
├─ data/
│   ├─ download.py    # [C] kagglehub download of fashion-product-images-small;
│   │                 #   locates styles.csv + images/, idempotent.
│   ├─ preprocess.py  # [C] Filter to top-10 subCategory classes, attach image
│   │                 #   paths, integer-encode labels, stratified 70/15/15.
│   └─ dataset.py     # [C] tf.data pipeline -> ((image, text), label) and the
│                     #   shared, train-only TextVectorization layer.
├─ models/
│   ├─ image_branch.py # [A] Frozen EfficientNetB0 -> GAP -> Dense(256).
│   ├─ text_branch.py  # [B] TextVectorization -> Embedding -> GRU(128) -> Dense(128).
│   └─ fusion.py       # [C] Concatenate(image, text) -> Dense(256) -> Dropout
│                     #   -> softmax. Reuses the two encoder builders verbatim.
├─ train.py           # [C] CLI: --model {image,text,fusion}; Adam + sparse CE,
│                     #   EarlyStopping; saves model + history + curves.
├─ evaluate.py        # [C] Test-set metrics (acc, macro P/R/F1) + confusion matrix.
└─ compare.py         # [C] Assembles the 3-way comparison table/chart and prints
                     #   the headline verdict: does fusion beat both baselines?
```

[A] = image branch (Person A), [B] = text branch (Person B), [C] = shared infra (Person C).

Per-file walkthrough

config.py — The single source of truth. Every path, hyperparameter, the class count (`N_CLASSES = 10`), and the random seed (`SEED = 42`) live here, so no magic numbers are scattered across modules and the report's "experimental setup" is a direct transcription of this file. It is deliberately import-safe without TensorFlow installed — the only TF use is inside `set_seeds()`, where the import is lazy — so the package imports on any machine while training happens on Colab. Also provides small helpers: `set_seeds()` (seeds Python/NumPy/TF), `ensure_dirs()`, and `save_class_list() / load_class_list()` so label encoding is identical across every run.

`data/download.py` — Fetches the Kaggle *Fashion Product Images (small)* dataset via `kagglehub` (no manual `kaggle.json` juggling on Colab). It is idempotent: `kagglehub` caches the download, and the resolved dataset root is recorded in `data/dataset_path.txt` so later stages never re-resolve the cache. Because different mirrors nest the files differently, it locates `styles.csv` by search and derives the `images/` directory relative to it, returning the `(styles_csv, images_dir)` pair the rest of the pipeline consumes.

`data/preprocess.py` — Turns the raw CSV into reproducible split manifests. It loads `styles.csv` (skipping the handful of malformed rows it's known for), drops rows missing an id/label/text, keeps the **top-10 most frequent** `subCategory` classes, attaches `images/{id}.jpg` paths (dropping rows whose image is absent), integer-encodes labels against a persisted ordered class list, optionally stratified-caps the size for faster Colab iteration, and produces a **stratified 70/15/15** split with the fixed seed. Each step is a small DataFrame-in/DataFrame-out function so it can be unit-tested on a synthetic frame without downloading the dataset. Outputs `train.csv` / `val.csv` / `test.csv`.

`data/dataset.py` — The `tf.data` input pipeline plus the shared text vectorizer. **Raw strings flow through the pipeline** — `TextVectorization` is the first layer of the text model, so tokenization happens inside the graph and every model variant sees the identical vectorizer with no leakage. `make_dataset()` is modality-aware: it yields `(image, label)`, `(text, label)`, or `((image, text), label)` depending on the variant, so the text-only branch never pays the JPEG decode/resize cost. The vectorizer is **adapted on the train split only** and its vocabulary persisted to `artifacts/text_vocab.txt`, so all three models reuse the same vocabulary across runs.

`models/image_branch.py` (**Person A**) — The image encoder: a `Input` → `EfficientNetB0(imagenet, include_top=False)` backbone, **frozen** (so its `BatchNorm` runs in inference mode, exactly what a fixed feature extractor wants), then `GlobalAveragePooling2D` → `Dense(256, relu)`. It exposes two functions: `build_image_encoder()` returns the 256-d encoding tensor for fusion to concatenate, and `build_image_model()` wraps that same encoder with a softmax head into the standalone image-only baseline — guaranteeing the baseline and the fusion model share an identical encoder.

`models/text_branch.py` (**Person B**) — The text encoder, mirroring the image branch's structure: `TextVectorization` → `Embedding(mask_zero=True)` → `GRU(128)` → `Dense(128, relu)`. The GRU returns only its **final hidden state**, which is used as the text encoding. The shared, train-adapted `TextVectorization` layer is **passed in** rather than rebuilt, so the text-only baseline and the fusion model tokenize identically. As with the image branch, it provides both a reusable `build_text_encoder()` and a standalone `build_text_model()`.

`models/fusion.py` (**Person C**) — The multi-modal model. It **reuses the exact encoder builders** from the image and text branches and differs from the baselines only in the fusion head: `Concatenate([image(256), text(128)])` → `Dense(256, relu)` → `Dropout(0.3)` → `Dense(n_classes, softmax)`. Reusing the encoders verbatim is what makes the 3-way comparison a fair, apples-to-apples test — only the presence of the second modality changes. The post-concat width and dropout are parameterized so the hyperparameter grid can sweep them.

`train.py` — The training entry point, `--model {image,text,fusion}`. It builds and compiles the requested variant (Adam, sparse categorical cross-entropy, accuracy), trains with

`EarlyStopping(restore_best_weights=True)` , and persists everything a graded run needs under `artifacts/<run-name>/` : the trained `model.keras` , `history.json` , a `metrics.json` summary (best val metrics, epoch count, and the exact hyperparameters used), and a `training_curves.png` of loss/accuracy. CLI flags expose the learning rate, dropout, fusion width, and an optional `--trainable-backbone` for the optional Stage-2 fine-tuning.

evaluate.py — Scores a trained variant on the **held-out test split**. It loads `model.keras` , runs predictions over the unshuffled test set (labels are read straight from the manifest and line up row-for-row, since `tf.data.map` is order-preserving — no double image decode), then computes accuracy, macro precision/recall/F1, and a full per-class report, writing `test_metrics.json` and a row-normalized `confusion_matrix.png` . These JSON files are the inputs to the comparison step.

compare.py — Assembles the **core deliverable**: it reads all three `test_metrics.json` files and emits `comparison.csv` , `comparison.md` (drop straight into the report), and a grouped bar chart `comparison.png` of accuracy and macro-F1. It then computes and prints the headline verdict — whether fusion's macro-F1 **beats** the best single modality, and by how much — which is the central claim of both the report and the presentation.